movie_industry_clustering

September 10, 2026

1 Movie Industry Data Analysis and Clustering

Portfolio version of a university data-analysis project

This project explores a movie-industry dataset using exploratory data analysis and **K-means clustering**.

The main questions are:

1. How are production budget, IMDb score, rating category, and gross revenue related?
2. Which genres tend to have higher typical gross revenue?
3. Can movies be grouped into interpretable clusters based on financial and rating characteristics?

The analysis is descriptive and exploratory. It does **not** make causal claims about what causes a movie to succeed.

1.1 1. Setup

Install the required packages once if necessary:

```
install.packages(c("tidyverse"))
```

[2]:
```
library(tidyverse)

theme_set(theme_minimal(base_size = 12))
```

```
  Attaching core tidyverse packages                tidyverse
2.0.0
  dplyr     1.1.4       readr     2.1.6
  forcats   1.0.1       stringr   1.6.0
  ggplot2   4.0.1       tibble    3.3.0
  lubridate 1.9.4       tidyr     1.3.2
  purrr     1.2.0
  Conflicts
tidyverse_conflicts()
  dplyr::filter() masks stats::filter()
  dplyr::lag()    masks stats::lag()
  Use the conflicted package
(<http://conflicted.r-lib.org/>) to force all conflicts to
become errors
```

1.2 2. Load and prepare the data

[3]:
```r
movies <- read_csv("movies.csv", show_col_types = FALSE)

glimpse(movies)

analysis_data <- movies |>
  transmute(
    name,
    year,
    genre,
    rating,
    score,
    budget,
    gross,
    r_rated = if_else(rating == "R", 1, 0)
  ) |>
  drop_na(score, rating, budget, gross)

tibble(
  original_rows = nrow(movies),
  analysis_rows = nrow(analysis_data),
  first_year = min(movies$year, na.rm = TRUE),
  last_year = max(movies$year, na.rm = TRUE)
)
```

```
Rows: 7,668
Columns: 15
$ name     <chr> "The Shining", "The Blue
Lagoon", "Star Wars: Episode V - The…
$ rating   <chr> "R", "R", "PG",
"PG", "R", "R", "R", "R",
"PG", "R", "PG", "P…
$ genre    <chr> "Drama", "Adventure",
"Action", "Comedy", "Comedy",
"Horror",…
$ year     <dbl> 1980, 1980, 1980,
1980, 1980, 1980, 1980, 1980,
1980, 1980, 1…
$ released <chr> "June 13, 1980 (United States)",
"July 2, 1980 (United States…
```

```
$ score    <dbl> 8.4, 5.8, 8.7,
7.7, 7.3, 6.4, 7.9, 8.2,
6.8, 7.0, 6.1, 7.3, 5…
$ votes    <dbl> 927000, 65000,
1200000, 221000, 108000, 123000,
188000, 33000…
$ director <chr> "Stanley Kubrick", "Randal
Kleiser", "Irvin Kershner", "Jim A…
$ writer   <chr> "Stephen King", "Henry De Vere
Stacpoole", "Leigh Brackett", …
$ star     <chr> "Jack Nicholson", "Brooke
Shields", "Mark Hamill", "Robert Ha…
$ country  <chr> "United Kingdom", "United
States", "United States", "United S…
$ budget   <dbl> 1.9e+07, 4.5e+06,
1.8e+07, 3.5e+06, 6.0e+06, 5.5e+05,
2.7e+07…
$ gross    <dbl> 46998772, 58853106,
538375067, 83453539, 39846344, 39754601,
…
$ company  <chr> "Warner Bros.", "Columbia
Pictures", "Lucasfilm", "Paramount …
$ runtime  <dbl> 146, 104, 124,
88, 98, 95, 133, 129,
127, 100, 116, 109, 114,…
```

A tibble: 1 × 4

original_rows	analysis_rows	first_year	last_year
<int>	<int>	<dbl>	<dbl>
7668	5424	1980	2020

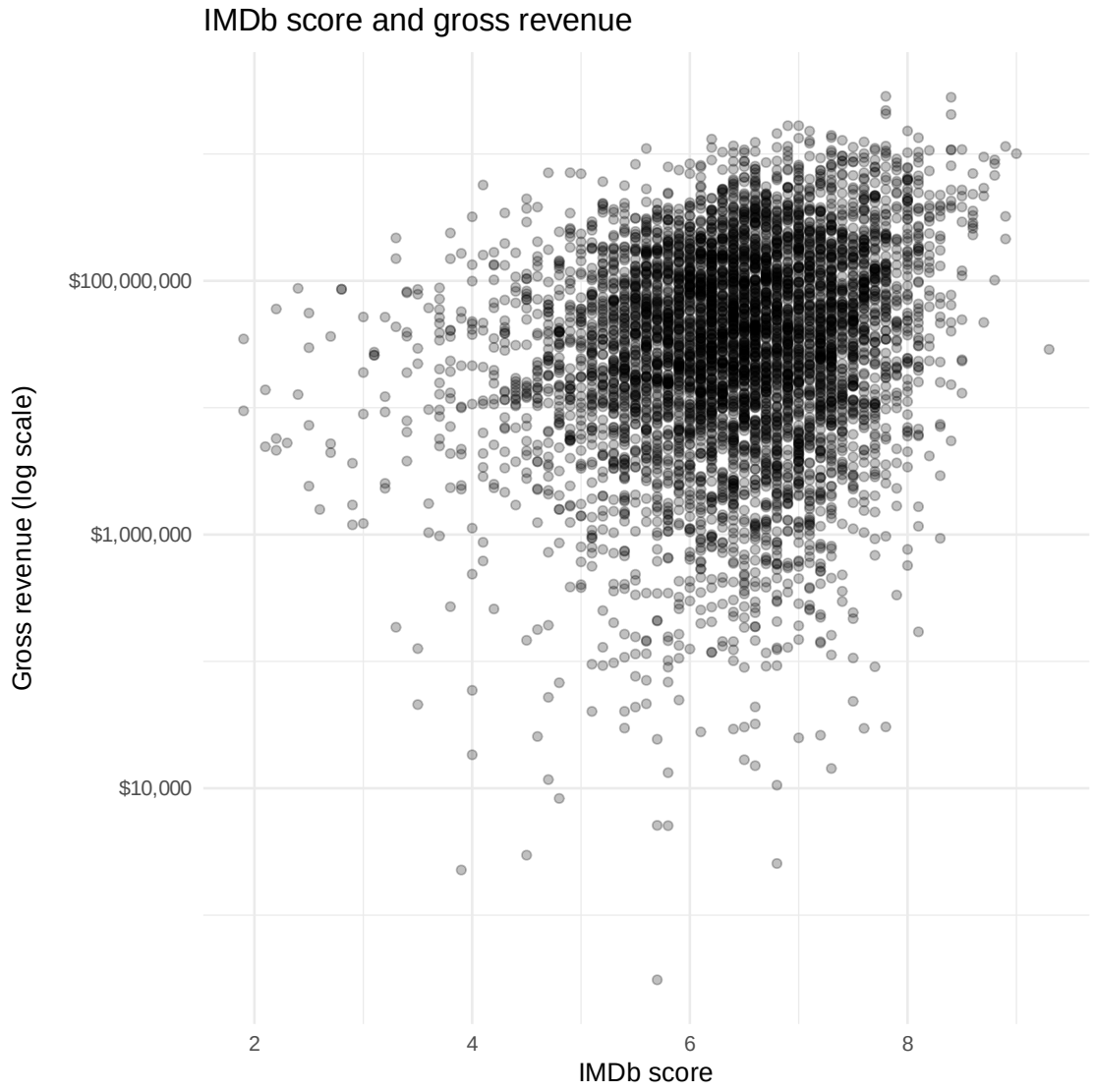
Only observations with non-missing values for the variables used in the analysis are removed. This avoids discarding a movie merely because an unrelated field (for example, company or writer) is missing.

The binary `r_rated` variable is used only as a compact rating feature for clustering. It intentionally loses information contained in the full rating categories, which is a limitation discussed later.

1.3 3. Exploratory data analysis

[4]:
```
ggplot(analysis_data, aes(x = score, y = gross)) +
  geom_point(alpha = 0.25) +
  scale_y_log10(labels = scales::label_dollar()) +
  labs(
```

```
    title = "IMDb score and gross revenue",
    x = "IMDb score",
    y = "Gross revenue (log scale)"
  )
```



[5]:
```
ggplot(analysis_data, aes(x = budget, y = gross)) +
  geom_point(alpha = 0.25) +
  scale_x_log10(labels = scales::label_dollar()) +
  scale_y_log10(labels = scales::label_dollar()) +
  labs(
    title = "Production budget and gross revenue",
    x = "Budget (log scale)",
    y = "Gross revenue (log scale)"
```

```
)
```

Gross revenue and production budget are strongly right-skewed, so logarithmic axes make the structure easier to inspect. The plots show association, not causation.

1.3.1 Gross revenue by genre

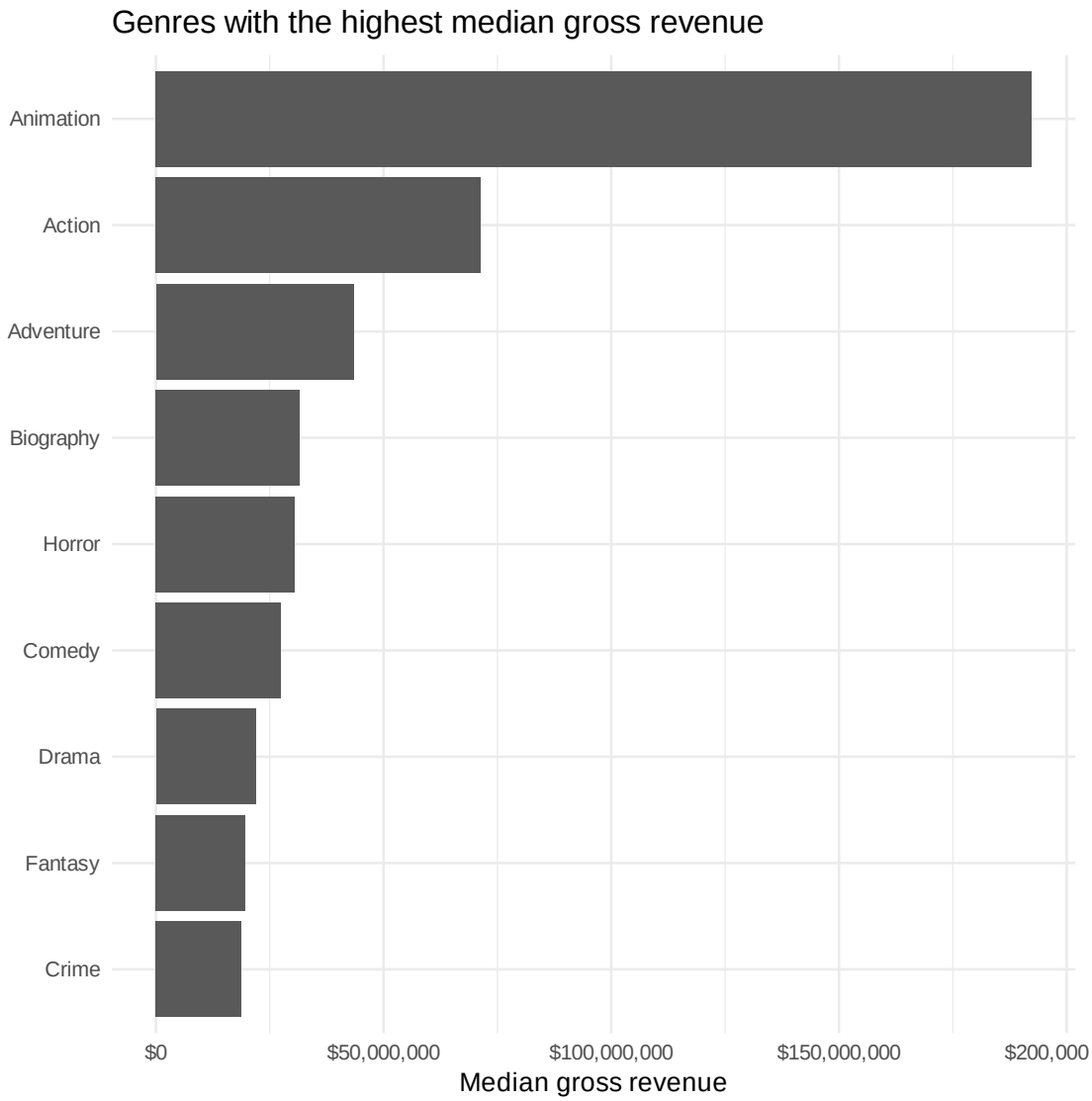
[6]:
```
genre_summary <- analysis_data |>
  group_by(genre) |>
  summarise(
    movies = n(),
    median_gross = median(gross),
    mean_gross = mean(gross),
```

```
    median_budget = median(budget),
    .groups = "drop"
  ) |>
  filter(movies >= 30) |>
  arrange(desc(median_gross))

genre_summary
```

A tibble: 9 × 5

genre <chr>	movies <int>	median_gross <dbl>	mean_gross <dbl>	median_budget <dbl>
Animation	277	192306508	281104365	7.0e+07
Action	1416	71335866	167931464	4.0e+07
Adventure	327	43411001	133268232	2.5e+07
Biography	312	31642680	61207107	2.0e+07
Horror	251	30553394	56816952	1.0e+07
Comedy	1496	27394834	59167659	1.7e+07
Drama	864	21920819	60300535	1.6e+07
Fantasy	41	19721741	39878698	9.0e+06
Crime	399	18753438	50169579	1.5e+07

[7]:
```
genre_summary |>
  slice_max(median_gross, n = 10) |>
  ggplot(aes(x = reorder(genre, median_gross), y = median_gross)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = scales::label_dollar()) +
  labs(
    title = "Genres with the highest median gross revenue",
    x = NULL,
    y = "Median gross revenue"
  )
```



Median gross revenue is reported alongside the mean because movie revenue is highly skewed and a small number of blockbusters can strongly affect averages.

1.4 4. Prepare features for clustering

[8]:
```
cluster_data <- analysis_data |>
  mutate(
    log_budget = log1p(budget),
    log_gross = log1p(gross)
  )

cluster_features <- cluster_data |>
  select(log_budget, log_gross, score, r_rated)
```

```
scaled_features <- scale(cluster_features)
```

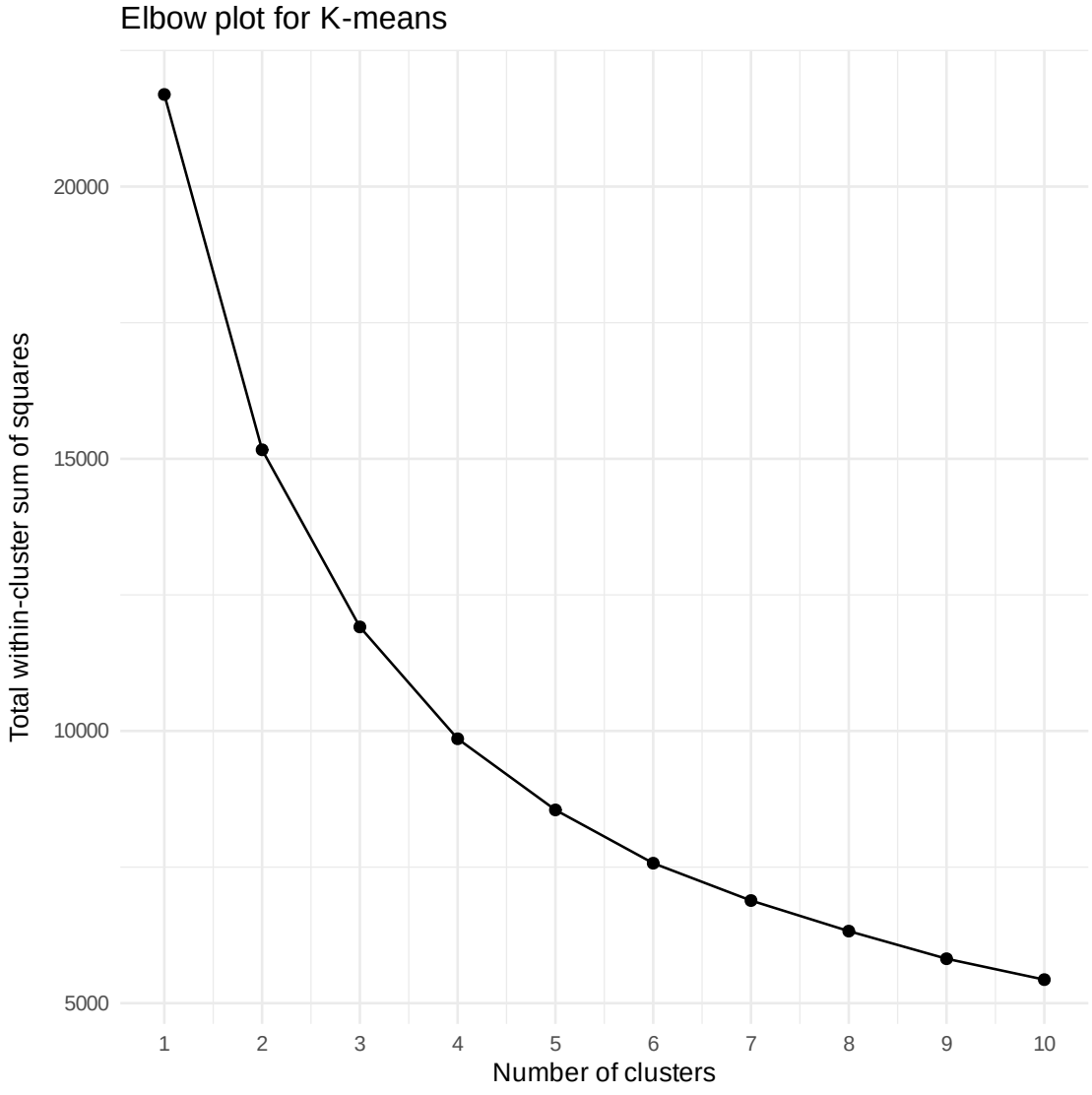
K-means is sensitive to feature scale. The clustering variables are therefore standardized before fitting the model. Budget and gross revenue are log-transformed first to reduce the influence of extreme values.

1.5 5. Choose the number of clusters

[9]:
```
set.seed(9999)

k_values <- 1:10

wss <- map_dbl(k_values, function(k) {
  kmeans(
    scaled_features,
    centers = k,
    nstart = 50
  )$tot.withinss
})

selection_table <- tibble(
  k = k_values,
  WSS = wss
)

selection_table

ggplot(selection_table, aes(x = k, y = WSS)) +
  geom_line() +
  geom_point(size = 2) +
  scale_x_continuous(breaks = k_values) +
  labs(
    title = "Elbow plot for K-means",
    x = "Number of clusters",
    y = "Total within-cluster sum of squares"
  )
```

A tibble: 10 × 2

k <int>	WSS <dbl>
1	21692.000
2	15165.330
3	11911.236
4	9857.148
5	8550.217
6	7571.620
7	6885.793
8	6324.170
9	5816.658
10	5433.331



The elbow plot is an exploratory model-selection tool rather than a formal proof that one value of

(k) is uniquely correct.

To keep the selection reproducible, the code below estimates the elbow as the point with the largest distance from the straight line joining the first and last points of the normalized WSS curve.

[10]:
```
x_norm <- (selection_table$k - min(selection_table$k)) /
          (max(selection_table$k) - min(selection_table$k))

y_norm <- (selection_table$WSS - min(selection_table$WSS)) /
          (max(selection_table$WSS) - min(selection_table$WSS))

distance_from_line <- (1 - x_norm) - y_norm

chosen_k <- selection_table$k[which.max(distance_from_line)]
cat("Selected number of clusters:", chosen_k, "\n")
```

```
Selected number of clusters: 4
```

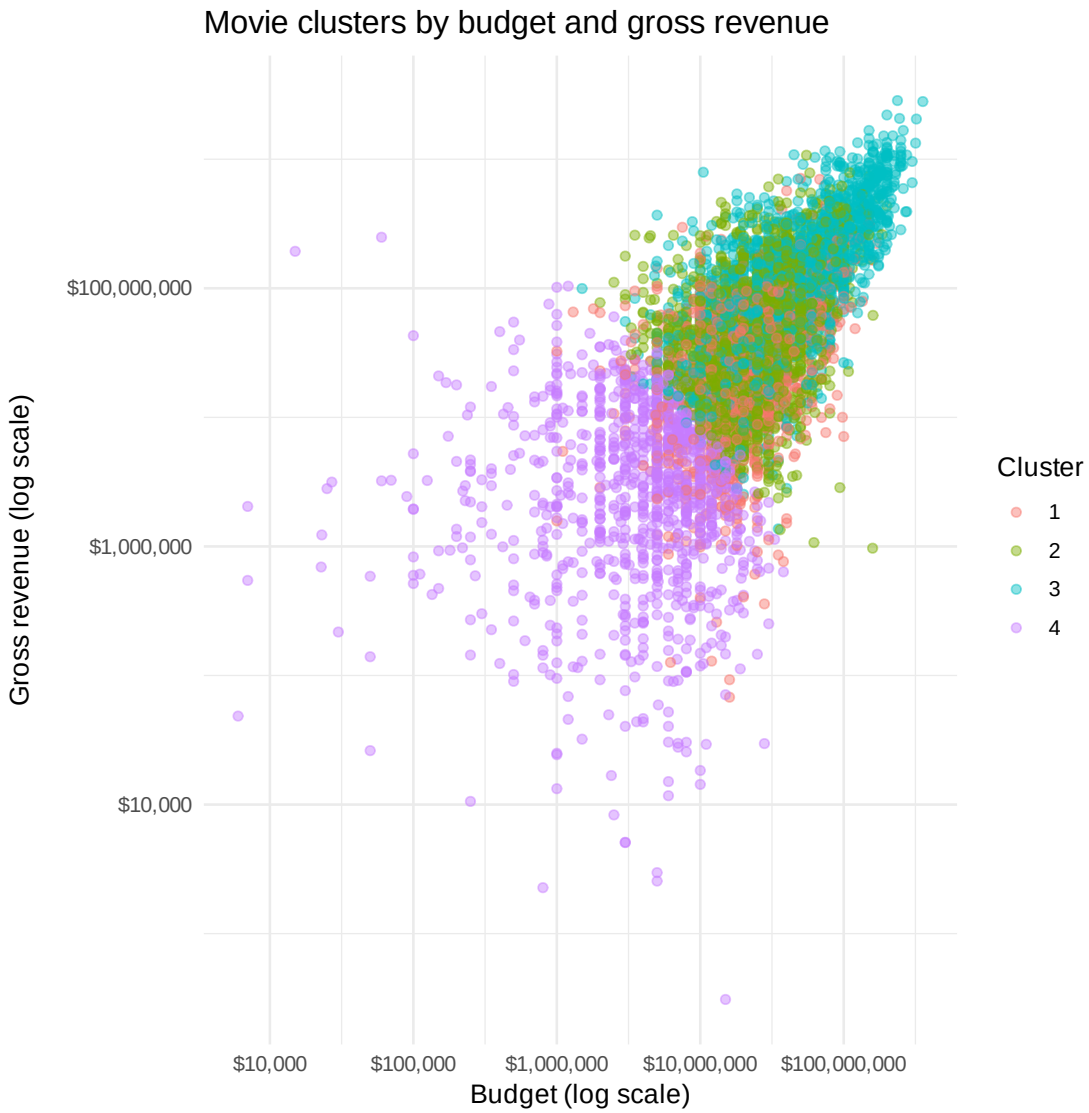
1.6 6. Fit and profile the K-means model

[11]:
```
set.seed(9999)

movie_kmeans <- kmeans(
  scaled_features,
  centers = chosen_k,
  nstart = 50
)

clustered_movies <- cluster_data |>
  mutate(cluster = factor(movie_kmeans$cluster))

cluster_summary <- clustered_movies |>
  group_by(cluster) |>
  summarise(
    movies = n(),
    median_budget = median(budget),
    median_gross = median(gross),
    mean_score = mean(score),
    share_r_rated = mean(r_rated),
    .groups = "drop"
  ) |>
  arrange(desc(median_gross))

cluster_summary
```

A tibble: 4 × 6

cluster <fct>	movies <int>	median_budget <dbl>	median_gross <dbl>	mean_score <dbl>	share_r_rated <dbl>
3	1662	4.3e+07	125240184	6.801625	0.00000000
2	1677	2.5e+07	45479110	6.657841	1.00000000
1	1038	2.0e+07	22555753	5.274085	0.05491329
4	1047	5.0e+06	3039587	6.424164	0.82425979

[12]:
```
ggplot(
  clustered_movies,
  aes(x = budget, y = gross, color = cluster)
) +
  geom_point(alpha = 0.45) +
  scale_x_log10(labels = scales::label_dollar()) +
  scale_y_log10(labels = scales::label_dollar()) +
  labs(
    title = "Movie clusters by budget and gross revenue",
    x = "Budget (log scale)",
    y = "Gross revenue (log scale)",
    color = "Cluster"
  )
```



1.6.1 Genre composition of each cluster

[13]:
```
cluster_genres <- clustered_movies |>
  count(cluster, genre, name = "movies") |>
  group_by(cluster) |>
  mutate(share = movies / sum(movies)) |>
  slice_max(share, n = 5, with_ties = FALSE) |>
  ungroup() |>
  arrange(cluster, desc(share))

cluster_genres
```

A tibble: 20 × 4

cluster <fct>	genre <chr>	movies <int>	share <dbl>
1	Comedy	441	0.42485549
1	Action	244	0.23506744
1	Drama	113	0.10886320
1	Adventure	103	0.09922929
1	Horror	46	0.04431599
2	Action	526	0.31365534
2	Comedy	362	0.21586166
2	Drama	270	0.16100179
2	Crime	226	0.13476446
2	Biography	109	0.06499702
3	Action	499	0.30024067
3	Comedy	375	0.22563177
3	Drama	240	0.14440433
3	Animation	220	0.13237064
3	Adventure	145	0.08724428
4	Comedy	318	0.30372493
4	Drama	241	0.23018147
4	Action	147	0.14040115
4	Crime	125	0.11938873
4	Horror	99	0.09455587

1.7 7. Interpretation

The cluster summary should be interpreted as a description of groups found by the algorithm, not as evidence that budget, rating, or IMDb score *causes* higher revenue.

The most useful comparisons are:

- differences in median budget and median gross revenue across clusters;
- whether high-revenue clusters also have higher IMDb scores;
- how the proportion of R-rated films varies between clusters;
- which genres are most common within each cluster.

Because gross revenue is itself included as a clustering feature, the clusters are designed partly around financial performance. They should therefore be viewed as a segmentation tool, not as a predictive model of future revenue.

1.8 8. Limitations

- K-means favors approximately spherical clusters in the feature space.
- The result depends on feature selection, transformations, scaling, and the chosen number of clusters.
- Reducing movie rating to `R` versus `non-R` discards information.
- Budget and gross figures may be inconsistently reported and are not inflation-adjusted.
- IMDb score is an audience rating, not an objective measure of quality.
- Associations in this dataset should not be interpreted causally.
- Revenue alone does not capture profitability because distribution, marketing, and other costs are not available.

1.9 9. Conclusion

This project demonstrates an exploratory data-analysis workflow combining data cleaning, visualization, feature transformation, standardization, and unsupervised learning.

The main value of K-means here is **segmentation**: it provides a compact way to compare groups of movies with different financial and rating profiles. The genre summaries complement the clustering analysis by showing how typical gross revenue varies across genres.

A natural next step would be to formulate a separate supervised-learning problem—for example, predicting log gross revenue or classifying films into financial-performance categories—and evaluate it on held-out data.

1.10 Data source

Daniel Grijalva, *Movie Industry* dataset, Kaggle:
https://www.kaggle.com/datasets/danielgrijalvas/movies